



Solana P-256, secp256k1, and SBF Program
Security Audit Report

Anza Technology

Final Audit Report: 4 September 2026

Table of Contents

[Overview](#)

[Background](#)

[Project Dates](#)

[Review Team](#)

[Coverage](#)

[Target Code and Revision](#)

[Supporting Documentation](#)

[Areas of Concern](#)

[Findings](#)

[General Comments](#)

[System Design](#)

[Code Quality](#)

[Documentation and Code Comments](#)

[Scope](#)

[Specific Issues & Suggestions](#)

[Suggestions](#)

[Suggestion 1: Enforce SIMD-565 Scalar Bounds Before Invoking P-256 Group Operations](#)

[About Least Authority](#)

[Our Methodology](#)

Overview

Background

Anza Technology has requested that Least Authority perform a security audit of Solana P-256, secp256k1, and the SBF Program.

Project Dates

- **August 19, 2026 - August 27, 2026:** Initial Code Review (*Completed*)
- **August 31, 2026:** Delivery of Initial Audit Report (*Completed*)
- **September 4, 2026:** Verification Review (*Completed*)
- **September 4, 2026:** Delivery of Final Audit Report (*Completed*)

Review Team

- Mirco Richter, Cryptography Researcher and Engineer
- Burak Atasoy, Project Manager
- Jessy Bissal, Technical Editor

Coverage

Target Code and Revision

For this audit, we performed research, investigation, and review of the solana-secp256r1 cryptographic primitives, the solana-secp256k1-verify library, and the standalone solana-secp256k1-program SBF signature-verification program, followed by issue reporting, along with mitigation and remediation instructions as outlined in this report.

The following code repositories were considered in scope for the review:

- The solana-secp256r1 crate in the anza-xyz/cryptography repository:
 - Repository URL: <https://github.com/anza-xyz/cryptography>
 - Crate URL: <https://github.com/anza-xyz/cryptography/tree/master/secp256r1>
- The secp256k1 library implementation and program for projects that rely on existing precompiles:
 - Repository URL: <https://github.com/solana-program/secp256k1>
 - Library URL: <https://github.com/solana-program/secp256k1/tree/main/secp256k1-verify>
 - Program URL: <https://github.com/solana-program/secp256k1/tree/main/program>

Specifically, we examined the following Git revisions for our initial review:

- <https://github.com/LeastAuthority/anza-cryptography/tree/master>
 - 6de114d1a38c88aed5de5aa7cf7d975067f2dd3
- <https://github.com/LeastAuthority/solana-program-secp256k1>
 - 7398e712639e6c2987e70f3d2bacf9a5f1dc97f

For the verification, we examined the following Git revisions:

- https://github.com/zz-sol/cryptography/tree/zz/secp256r1_audit
 - 342ef7190c7bb0cb13b7f895ca58f67f36672ca3

For the review, these repositories were cloned for use during the audit and for reference in this report:

- solana-program-secp256k1:
<https://github.com/LeastAuthority/solana-program-secp256k1>
- anza-cryptography:
<https://github.com/LeastAuthority/anza-cryptography>

All file references in this document use Unix-style paths relative to the project's root directory.

In addition, any dependency and third-party code, unless specifically mentioned as in scope, were considered out of scope for this review.

Supporting Documentation

The following documentation was available to the review team:

- secp256r1 curve implementation for the secp256r1 syscall (SIMD-565) specification:
<https://github.com/solana-foundation/solana-improvement-documents/blob/main/proposals/0565-secp256r1-syscalls.md>
- Website:
<https://www.anza.xyz>

In addition, this audit report references the following documents:

- NIST SP 800-186: Elliptic Curve Domain Parameters:
<https://csrc.nist.gov/pubs/sp/800/186/final>
- SIMD-565: secp256r1 Curve Syscalls:
<https://github.com/solana-foundation/solana-improvement-documents/blob/5b1a323c0c73755ec97a8be3f9992f1502f8cf3f/proposals/0565-secp256r1-syscalls.md>
- Agave Generalized Curve and secp256k1 Syscall Implementation:
<https://github.com/anza-xyz/agave/blob/2a165e7a90af75c76426d1e031ed0284211d5d1e/syscalls/src/lib.rs>
- Aeneas Rust Verification Toolchain:
<https://github.com/AeneasVerif/aeneas>
- Charon Rust Translation Frontend:
<https://github.com/AeneasVerif/charon>
- Lean 4 Theorem Prover:
<https://lean-lang.org>
- Mathlib4:
<https://github.com/leanprover-community/mathlib4>
- cvc5 SMT Solver:
<https://cvc5.github.io>

Areas of Concern

Our investigation focused on the following areas:

- Correctness of the implementation;
- Vulnerabilities within each component and whether the interaction between the components is secure;
- Denial of Service (DoS) and other security exploits that would impact the intended use or disrupt the execution;
- Protection against malicious attacks and other ways to exploit;

- Inappropriate permissions and excess authority;
- Data privacy, data leaking, and information integrity; and
- Anything else as identified during the initial analysis phase.

Findings

General Comments

The Anza [cryptography](#) repository contains Rust cryptographic libraries and syscall-facing primitives for the Solana ecosystem, covering several elliptic curves, signature schemes, proof systems, and CPU/GPU-accelerated implementations. The separate [secp256k1](#) project provides reusable verification logic and a Solana SBF program for authenticating Ethereum-compatible recoverable ECDSA signatures.

Our audit was limited to the `solana-secp256r1` crate, the `solana-secp256k1-verify` crate, and the accompanying program crate. `solana-secp256r1` implements NIST P-256 field, scalar, and group operations intended to support proposed native Solana syscalls and applications such as WebAuthn and passkeys. `solana-secp256k1-verify` provides configurable signature recovery and verification, while the program crate exposes a strict on-chain path that hashes messages with Keccak-256, recovers the signer, and validates the resulting EVM address. The P-256 and secp256k1 components operate independently and serve distinct Solana use cases.

System Design

Our team reviewed `secp256r1` and `secp256k1` and found them to be well-designed, with high-quality Rust code and clear consideration for security. Additionally, we identified no general areas of weakness or recurring patterns of concern.

During our review, we compared the `solana-secp256r1` implementation against SIMD-565, the NIST P-256 specification, reference implementations, and relevant Agave behavior. We traced field and scalar arithmetic through point parsing, validation, encoding, group operations, scalar multiplication, and multiscalar multiplication. We also reviewed canonical encoding, endianness, exceptional cases, resource behavior, and the formal proof boundary.

For `solana-secp256k1-verify`, we traced hashing, signature parsing, recovery identifiers, signature malleability policies, public key recovery, and public key or EVM address matching. For the program crate, we reviewed instruction framing, complete message binding, account handling, Keccak verification, runtime integration, and failure behavior. We also assessed feature configurations and exported interfaces.

Across the scope, we combined manual review with differential, boundary, randomized, and adversarial testing in debug and release configurations. We also examined panic resistance, allocation behavior, performance, and resource consumption. In addition, our team evaluated observations for attacker control, reachability, and concrete impact under Solana's execution model.

As a supplemental activity, we conducted an experimental formal verification exercise on selected parts of the `secp256r1` implementation. We translated the relevant Rust functions into Lean and checked key correctness properties against Mathlib's finite-field and elliptic-curve definitions. Our team documented the assumptions, trusted components, and uncovered areas in a dedicated proof boundary, which also informed the manual review. The corresponding code and documentation are available [here](#).

Dependencies

Running `cargo audit` revealed no dependency concerns.

Code Quality

We performed a manual review of the repositories in scope and found the code to be of high quality, well organized, and aligned with development best practices.

Tests

The repositories in scope include sufficient test coverage, with happy-path, boundary, and adversarial tests covering P-256 arithmetic and encoding, secp256k1 recovery and verification, and program instruction handling using valid and malformed inputs.

Documentation and Code Comments

The project documentation clearly outlines the project's purpose and sufficiently describes the system's intended functionality. Additionally, the codebase includes descriptive comments, which aid in understanding the intended behavior of the relevant components.

Scope

The scope of this review was sufficient and included all security-critical components.

Specific Issues & Suggestions

We list the issues and suggestions identified during the review in descending order of severity. In most cases, remediation of an issue is preferable, but mitigation is suggested as another option for cases where a trade-off could be required.

ISSUE / SUGGESTION	SEVERITY	STATUS
Suggestion 1: Enforce SIMD-565 Scalar Bounds Before Invoking P-256 Group Operations	Informational	Resolved

Suggestions

Suggestion 1: Enforce SIMD-565 Scalar Bounds Before Invoking P-256 Group Operations

Location

[secp256r1/src/group.rs#L394-L469](#)

[secp256r1/src/scalar.rs#L53-L87](#)

[solana-improvement-documents/proposals/0565-secp256r1-syscalls.md#L132-L185](#)

Synopsis

The raw P-256 group multiplication APIs accept every 256-bit scalar, including values greater than or equal to the group order n . Group arithmetic consequently produces the result corresponding to the scalar modulo n . SIMD-565 instead requires the syscall to reject such inputs.

The supplied code therefore does not independently enforce this syscall requirement. A conformance issue would arise if the separately implemented syscall adapter passed attacker-controlled scalar bytes directly to these APIs without first applying the crate's strict scalar parser.

Mitigation

We suggest that integrators parse every untrusted scalar using `Scalar::from_bytes`, reject unsuccessful parsing, and pass only its canonical big-endian representation to the group APIs. For multiscalar multiplication (MSM), all points and scalars should be validated before computation or output. A checked interface accepting the validated `Scalar` type could further reduce the risk of incorrect integration.

Status

The Anza team has resolved this issue in [PR#118](#) by rejecting noncanonical scalars before group multiplication and MSM computation.

Verification

Resolved.

About Least Authority

Founded in 2011 and based in Berlin, Germany, since 2016, Least Authority provides security consulting for privacy-preserving technologies and decentralized systems. We are committed to transparency, openness, and advancing secure digital infrastructure.

We believe that people have a fundamental right to privacy and that secure solutions enable people to use the internet and other connected technologies more freely. Our security consulting services help teams make their solutions more resistant to unauthorized access and unintended system manipulation. We support teams from the design phase through production launch and beyond.

Our team has experience reviewing code for security vulnerabilities and specific attack vectors in multiple languages, including Rust, Go, Python, C, C++, Solidity, Cairo, Circom, Noir, Haskell, OCaml, TypeScript, Swift, ZoKrates, and others. We have reviewed implementations of cryptographic protocols and distributed system architecture in blockchains, payments, smart contracts, zero-knowledge protocols, and consensus protocols. Additionally, we use various tools to scan code and networks and build custom tools as necessary.

For more information about our security consulting, please visit:

<https://leastauthority.com/security-consulting>.

Our Methodology

We use a transparent review process that provides the project team visibility into our findings, assumptions, and remediation considerations throughout the audit. The goals of our security audits are to improve the quality of the systems we review and support sufficient remediation to help protect users. The following is the methodology we use in our security audit process.

Code Review

In manually reviewing all of the code, we identify potential issues, including those related to code logic, error handling, cryptographic implementations, and correctness with respect to specifications, standards, and best practices. We also evaluate areas where more defensive programming could reduce the risk of future mistakes and accelerate future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

When appropriate, we also use artificial intelligence (AI) to support the efficiency of our security audits without reducing rigor or reliability. AI-assisted outputs are carefully guided, reviewed, and validated by our team so that conclusions remain accurate and defensible. This supports stronger audit decisions, not only faster review.

Vulnerability Analysis

Our audit techniques include manual code analysis, user interface interaction, and white box penetration testing, supplemented by various tools, including AI, that support the code review process. Discussions with the development team help clarify their vision for the software. Installing and using the relevant software helps us understand user interactions and roles, while also informing our analysis of threat models and attack surfaces. Design documentation, relevant papers, other audit results, similar projects, source code dependencies, and open issue tickets may also be reviewed to inform the analysis. We

identify potential vulnerabilities, and, when appropriate, create issue entries. For each entry, we follow the issue investigation and remediation process described below.

Documenting Results

We follow a conservative and transparent process for analyzing potential security vulnerabilities and supporting successful remediation. Whenever a potential issue is identified, we raise it with the client through a dedicated secure communication channel to discuss its validity and impact, to the extent these can be determined at that stage. This process allows potentially relevant findings to be reviewed with the client early, before they are documented in greater detail in the initial audit report. For each confirmed issue, we assess impact and feasibility and use those ratings to determine the overall severity, following the [OWASP Risk Severity Matrix](#).

Suggested Solutions

We identify immediate and comprehensive mitigations available to live deployments and then suggest remediation requirements for future releases. When code is already deployed in production, we include mitigation strategies in the report to provide interim support until appropriate remediation is available. The mitigation and remediation recommendations should be reviewed by the development team and deployment engineers. After delivering the Initial Audit Report, we remain available to support implementation of the proposed mitigations and remediations before the verification review.

Development teams should address findings in an order and with an urgency that prioritizes the security of their users. Findings should be addressed as soon as possible, but remediation should be carried out pragmatically and should not cause unnecessary disruption to project timelines.

Although issues must be handled on a case-by-case basis, we work toward a resolution timeline that balances the impact on users and the needs of the project team.

Resolutions & Publishing

Once the findings are comprehensively addressed, we complete a verification review to assess whether the issues and suggestions have been sufficiently addressed. When this analysis is complete, we update the report and provide a Final Audit Report that can be published in full. If any critical issues remain, we recommend that the Final Audit Report not be published and that users and other stakeholders be alerted to the impact. We encourage all findings to be addressed and the Final Audit Report to be shared publicly to support transparency and advance security knowledge within the industry.